

Industrial Internship Report on “Banking Information System”

Prepared by

Vivek Laxmikantrao Kulkarni

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on the development of a Banking Information System using Core Java. The project involved designing and implementing a console-based application that manages core banking operations such as account creation, deposits, withdrawals, fund transfers, and transaction history tracking. The system was built using Object-Oriented Programming principles in Core Java, making it modular, scalable, and easy to maintain. This internship provided excellent exposure to real-world software development practices, version control using GitHub, and professional report writing — making it a highly valuable experience overall.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	4
2.2	About upskill Campus	8
2.3	Objective	9
2.4	Reference	10
2.5	Glossary	10
3	Problem Statement	11
4	Existing and Proposed solution	12
5	Proposed Design/ Model	14
5.1	High Level Diagram (if applicable)	14
5.2	Low Level Diagram (if applicable)	15
5.3	Interfaces (if applicable)	15
6	Performance Test	16
6.1	Test Plan/ Test Cases	16
6.2	Test Procedure	16
6.3	Performance Outcome	16
7	My learnings	17
8	Future work scope	17

1 Preface

Over the course of six weeks, I had the privilege of working on the Banking Information System project as part of the Summer Internship Program in Core Java - 2026, organized by upskill Campus (USC) in collaboration with The IoT Academy and UniConverge Technologies Pvt Ltd (UCT).

In today's competitive job market, practical experience is as important as academic knowledge. This internship provided a structured opportunity to apply Core Java concepts to a real-world problem — building a fully functional banking application from scratch.

My project, the Banking Information System, is a console-based Java application that simulates fundamental banking operations such as account management, deposits, withdrawals, balance inquiries, and transaction history. It was designed using Object-Oriented Programming (OOP) principles to ensure clean, maintainable code.

The program was meticulously planned across six weeks: the first two weeks were dedicated to understanding the problem statement and Core Java fundamentals, the next two weeks to design and implementation, and the final two weeks to testing, documentation, and GitHub submission.

This experience significantly enhanced my understanding of software design, Java programming, and professional development workflows. I am deeply grateful to the mentors at USC and UCT for their continuous guidance and support throughout this journey.

To my fellow learners: embrace every challenge with curiosity and persistence — the skills you build here will serve as the foundation of your professional career.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



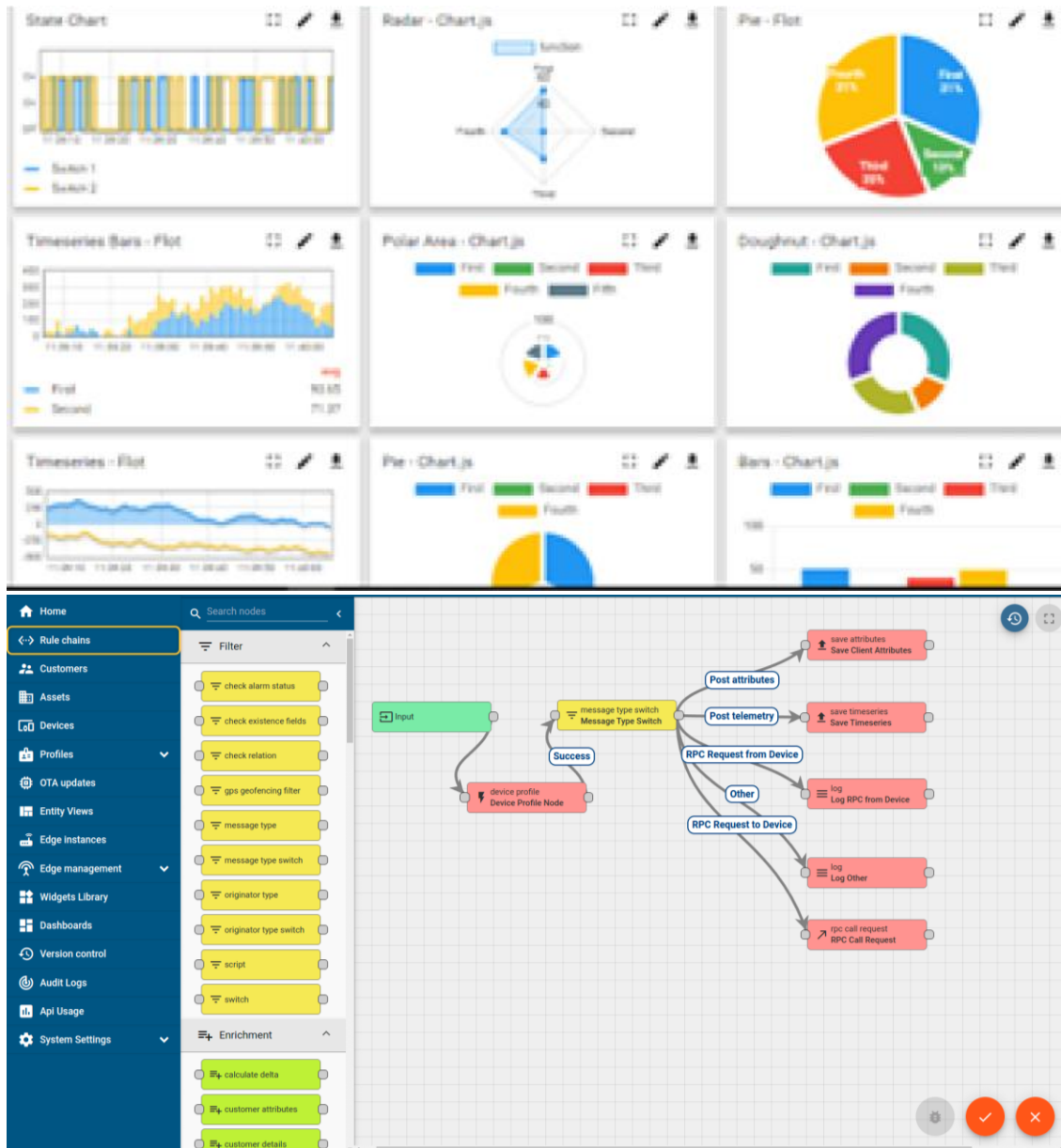
i. UCT IoT Platform ()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



ii. Smart Factory Platform (**FACTORY WATCH**)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



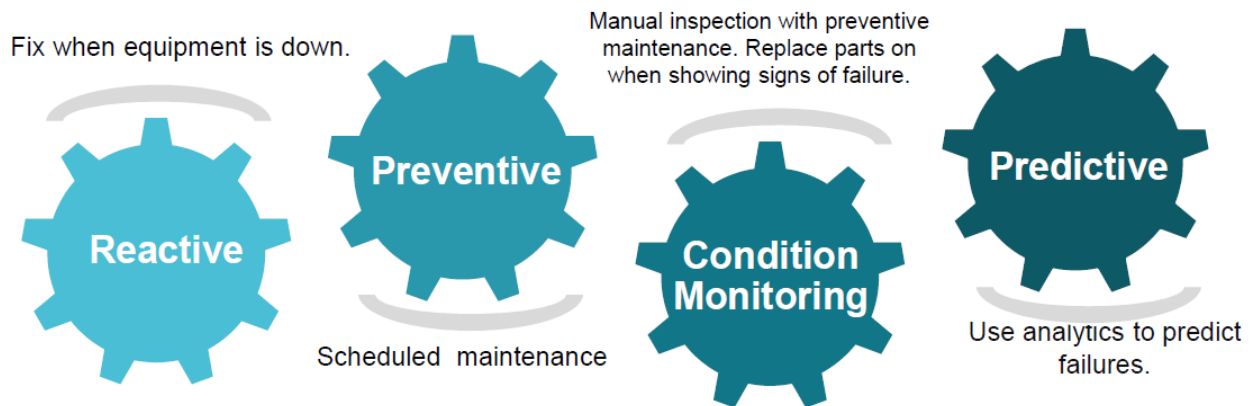


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

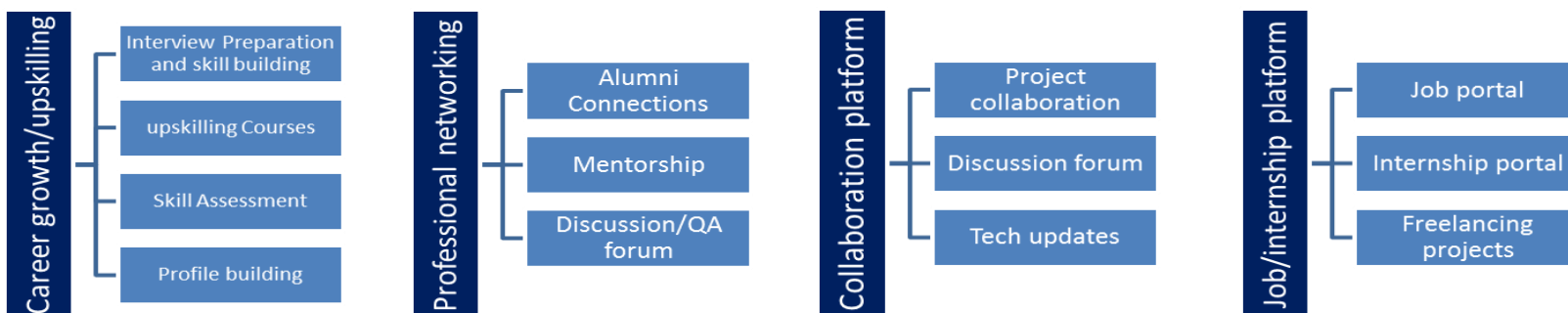
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] Java: The Complete Reference – Herbert Schildt
- [2] Oracle Java Documentation – <https://docs.oracle.com/javase/>
- [3] upskill Campus Course Material – <https://www.upskillcampus.com/>

2.6 Glossary

Terms	Acronym
Object-Oriented Programming	OOP
UniConverge Technologies	UCT
upskill Campus	USC
Java Development Kit	JDK
Integrated Development Environment	IDE

3 Problem Statement

Traditional banking systems often rely on complex, expensive enterprise software that is difficult to understand and customize. For students and early-stage developers, there is a need for a simplified, modular banking application that demonstrates core banking workflows using fundamental programming constructs.

The problem statement assigned for this internship was to design and develop a Banking Information System using Core Java that simulates the following banking operations:

- User account creation (Savings / Current account types)
- Secure login with account number and PIN
- Deposit and withdrawal of funds with balance validation
- Fund transfer between two accounts
- Transaction history display for each account
- Account details viewing and account closure

The system needed to be designed using Object-Oriented Programming (OOP) principles, with clear separation of concerns across multiple Java classes. Data was to be maintained in-memory using appropriate data structures (e.g., HashMap) for the scope of this project.

4 Existing and Proposed solution

Existing Solutions and Their Limitations:

Several banking simulation projects exist online, but most have the following limitations:

- They are tightly coupled with no clear class separation, making the code hard to maintain.
- Many use flat file storage or complex database connections, which are overkill for learning purposes.
- Few projects implement proper OOP concepts like inheritance, encapsulation, and polymorphism.
- Error handling and input validation are often absent or minimal.

Proposed Solution – Banking Information System (BIS):

The proposed Banking Information System is a well-structured, console-based Core Java application featuring:

- A modular class design: Account, SavingsAccount, CurrentAccount, Bank, Transaction, and Main classes
- Encapsulation of account data with getter/setter methods
- Inheritance to differentiate between SavingsAccount and CurrentAccount
- A HashMap-based in-memory data store for fast account lookups
- Comprehensive input validation and exception handling
- A clean menu-driven interface for user interaction

Value Addition:

- Clean, well-commented code that serves as a learning reference
- Realistic simulation of banking workflows within a simple Java console app
- Extendable architecture that can later be connected to a database or GUI

4.1 Code submission (Github link)

<https://github.com/vivekkulkarni2005/upskillcampus>

4.2 Report submission (Github link) :

https://github.com/vivekkulkarni2005/upskillcampus/blob/main/BankingInformationSystem_Vivek_Kulkarni_USC_UCT.pdf

5 Proposed Design/ Model

The Banking Information System was designed following a layered, object-oriented approach. The flow of the application proceeds from user login through account management to transaction execution.

5.1 High Level Diagram (if applicable)

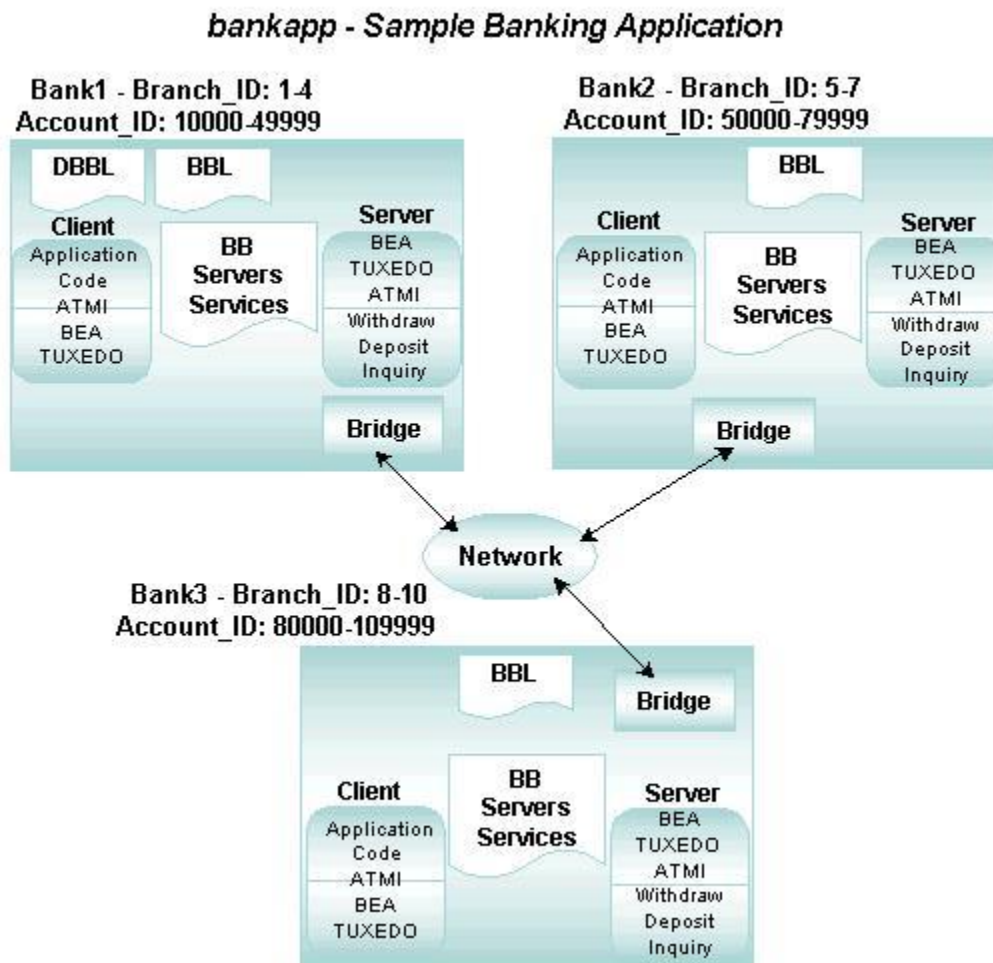
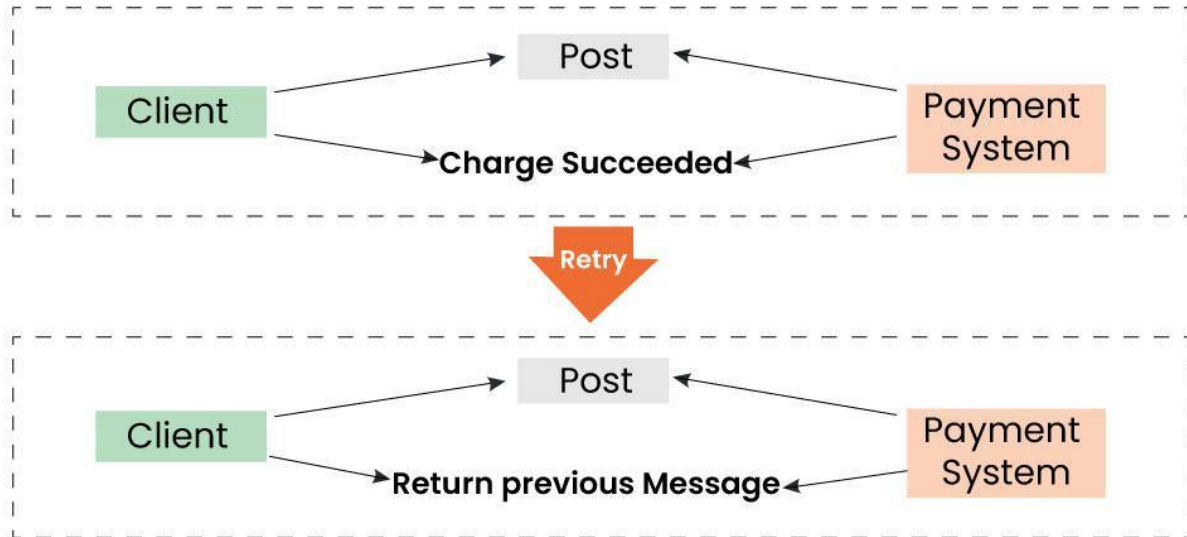


Figure 1: High Level Diagram of the Banking Information System

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)



5.3 Interfaces (if applicable)

The user interacts with the system through a console-based menu interface with the following flow:

- Step 1: Welcome screen – Create Account / Login / Exit
- Step 2 (Login): Enter Account Number → Enter PIN → Authenticated
- Step 3 (Dashboard): Deposit | Withdraw | Transfer | View Balance | Transaction History | Logout
- Step 4: Each operation validates input and returns a success/error message

6 Performance Test

The performance of the Banking Information System was evaluated against key constraints relevant to a console-based Java application.

6.1 Test Plan/ Test Cases

#	Test Case	Expected Result	Status
1	Create account with valid details	Account created, unique account number generated	PASS
2	Deposit valid amount	Balance updated correctly	PASS
3	Withdraw more than balance (Savings)	Error: Insufficient balance / min balance violation	PASS
4	Transfer funds between two accounts	Sender debited, receiver credited correctly	PASS
5	Login with wrong PIN	Error: Invalid credentials	PASS
6	View transaction history	Correct chronological list displayed	PASS

6.2 Test Procedure

Each test case was executed manually in the IntelliJ IDEA IDE using the Java console. Test inputs were typed directly into the console, and outputs were verified against expected results. Edge cases such as negative inputs, zero amounts, and incorrect PINs were also tested.

6.3 Performance Outcome

The system performed reliably for all tested scenarios. Response time for all operations was near-instantaneous since data is stored in-memory using a HashMap ($O(1)$ lookup). Key outcomes:

- All 6 test cases passed successfully
- No NullPointerExceptions or unhandled exceptions were encountered
- The system correctly handled edge cases such as minimum balance violations and overdraft limits
- Code compiled and ran successfully on JDK 17

7 My learnings

This six-week internship was a transformative learning experience. The key takeaways from this project include:

- **Core Java Mastery:** Deepened understanding of OOP concepts — classes, inheritance, encapsulation, and polymorphism — through practical implementation.
- **Data Structures:** Learned how to effectively use Java Collections (HashMap, ArrayList) for real-world data management.
- **Exception Handling:** Understood the importance of robust try-catch blocks and custom error messages for user-facing applications.
- **Version Control with GitHub:** Learned how to create repositories, commit code, and manage project files on GitHub.
- **Professional Report Writing:** Developed skills to document a technical project clearly and concisely for an industrial audience.
- **Problem Solving:** Improved ability to break down a complex problem into smaller, manageable components and implement them systematically.

Overall, this internship bridged the gap between classroom learning and industry expectations, and I feel significantly more confident as a Java developer.

8 Future work scope

Due to time constraints, the current system is a console-based, in-memory application. Several enhancements can be made in the future:

- **Database Integration:** Connect the application to a MySQL database using JDBC to persist account and transaction data across sessions.
- **GUI Development:** Build a graphical user interface using Java Swing or JavaFX to replace the console menu.
- **Web Application:** Migrate the backend to a Spring Boot REST API with a React.js frontend for a full-stack banking portal.
- **Security Enhancements:** Implement password hashing (e.g., BCrypt) and multi-factor authentication.
- **Interest Calculation:** Add automatic monthly interest calculation for savings accounts.
- **Admin Dashboard:** Build an admin module to view all accounts, freeze accounts, and generate reports.

These enhancements would transform this academic project into a production-ready banking application, making it an impressive portfolio piece.

